

4. 그래프의 표현

1) 인접행렬

2) 인접리스트

4-1) 인접행렬

- 인접행렬 (Adjacency Matrix)

- 그래프 $G=(V, E)$ 에서 $|V|=n$ 일 때 $n \times n$ 행렬로 나타내는 방법

- 그래프 $G=(V, E)$ 에 대한 $V=[a_{ij}]$ 의 각 원소

$$a_{ij}=a_{ji}=\begin{cases} 1 & (v_i, v_j) \in E \\ 0 & otherwise \end{cases}$$

- $G=(V, E)$

$$V=\{a, b, c, d\}$$

$$E=\{(a, b), (a, d), (b, c), (c, d)\}$$

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	0	1	0	1
<i>b</i>	1	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	0

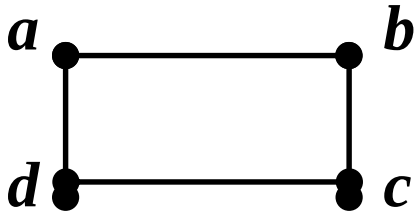
- 대칭행렬 (다중 그래프 표현에 한계)

- 루프가 있을 때 대각원소 값이 1

4-2) 인접리스트

- 인접리스트(Adjacency List)

- 그래프 $G=(V, E)$ 를 구성하는 각 꼭짓점들을 연결리스트(Linked List)로 표현한 것



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	0	1	0	1
<i>b</i>	1	0	1	0
<i>c</i>	0	1	0	1
<i>d</i>	1	0	1	0

<i>a</i>	→ <i>b</i> → <i>d</i> <i>null</i>
<i>b</i>	→ <i>a</i> → <i>c</i> <i>null</i>
<i>c</i>	→ <i>b</i> → <i>d</i> <i>null</i>
<i>d</i>	→ <i>a</i> → <i>c</i> <i>null</i>

5. 그래프의 활용

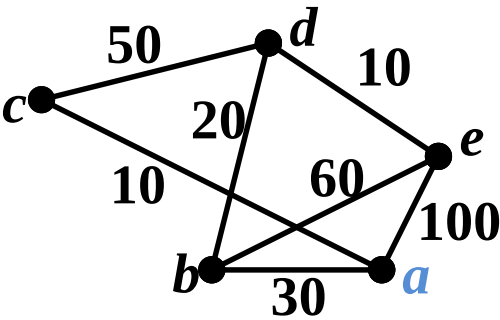
- 1) 최단경로 문제
- 2) 깊이 우선 탐색
- 3) 너비 우선 탐색

5-1) 최단경로 문제-1

- 최단경로문제(Shortest Path Problem)
 - $|E| > 0$ 인 그래프 $G=(V, E)$ 에서 꼭짓점 $v_1, v_2 \in V$ 간의 가장 짧은 거리의 경로를 찾는 문제
 - 출발점(Source): 경로의 시작점
 - 도착점(Destination): 경로의 목적지
 - $D[v_i]$: 시작점 v_1 에서 각 꼭짓점 v_i 까지 최단경로
 S (탐색 전 최단경로 집합) $= \{v_1\}$
 V (남은 꼭짓점) $= \{v_2, v_3, \dots, v_n\}$
 $C[v_i, v_j] = C_{v_i v_j}$: 꼭짓점 v_i 로부터 v_j 까지 거리
 - $C[v_i, v_i] = 0, C[v_i, v_j] = \infty$ (경로가 없을 때)

5-1) 최단경로 문제-2

단계	탐색 전	D[b]	D[c]	D[d]	D[e]	꼭짓점	탐색 후
①	S={a} V={b,c,d,e}	30	10	∞	100	c (a-c)	S={a,c} V={b,d,e}
②	S={a,c} V={b,d,e}	30	10	60	100	b (a-b)	S={a,c,b} V={d,e}
③	S={a,c,b} V={d,e}	30	10	50	90	d (a-b-d)	S={a,c,b,d} V={e}
④	S={a,c,b,d} V={e}	30	10	50	60	e (a-b-d-e)	S={a,c,b,d,e} V={ }



5-1) 최단경로 문제-3

• Dijkstra Algorithm

$D[v_i]$: 시작점 v_1 로부터 각 꼭짓점 v_i 의 최단경로
 $C[v_i, v_i]=0$, $C[v_i, v_j]=\infty$ (경로 없을 때), $C[v_i, v_j]=C_{v_i v_j}$

```
void Dijkstra( ) {  
     $S=\{v_1\}$ ;  
     $V=\{v_2, v_3, \dots, v_n\}$ ;  
    for ( $i=2$ ;  $i \leq n$ ;  $i++$ )  $D[v_i]=C[v_1, v_i]$ ;  
    for ( $i=1$ ;  $i \leq n-1$ ;  $i++$ ) {  
         $D[v_m]$ 이 최소면  $V-S$ 에서  $v_m$  선택;  $v_m$ 을  $S$ 에 추가;  
        for ( $V-S$ 에 포함된 모든 꼭짓점)  
             $D[v_n]=\min(D[v_n], D[v_m]+C[v_m, v_n])$ ;  
    }  
}
```

5-2) 깊이 우선 탐색-1

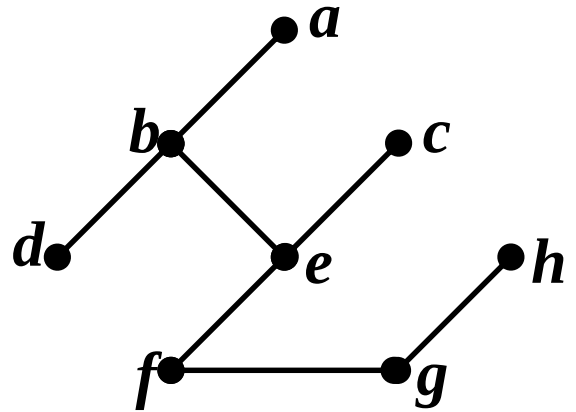
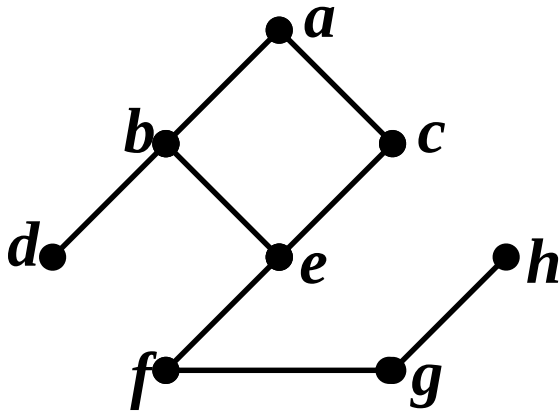
- **깊이 우선 탐색(DFS: Depth First Search)**

- ① 시작점 v 를 탐색한다.
- ② 꼭짓점 v 에 인접한 꼭짓점들 중 탐색되지 않은 꼭짓점 v_{sub} 를 탐색한다.
- ③ 꼭짓점 v_{sub} 를 v 로 하여 ②를 반복한다.
- ④ 더 이상 탐색되지 않은 꼭짓점이 없으면 바로 이전에 탐색한 꼭짓점을 v 로 하여 ②와 ③을 반복한다.
- ⑤ 그래프의 모든 꼭짓점을 탐색할 때까지 ④를 반복한다.

5-2) 깊이 우선 탐색-2

- 깊이 우선 탐색(DFS) $|V|=8$

- $a > b > d < (b) > e > f > g > h < (g) < (f) < (e) > c$



- $a-b-d-e-f-g-h-c$

5-3) 너비 우선 탐색-1

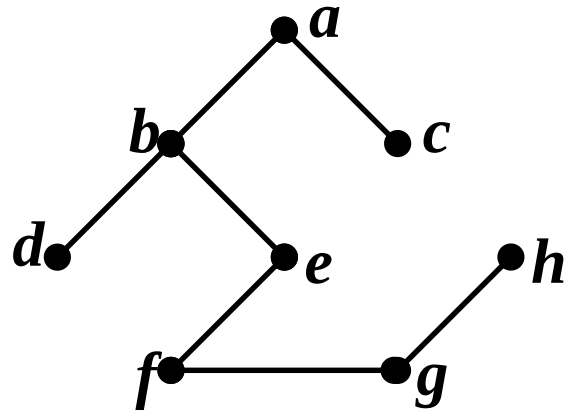
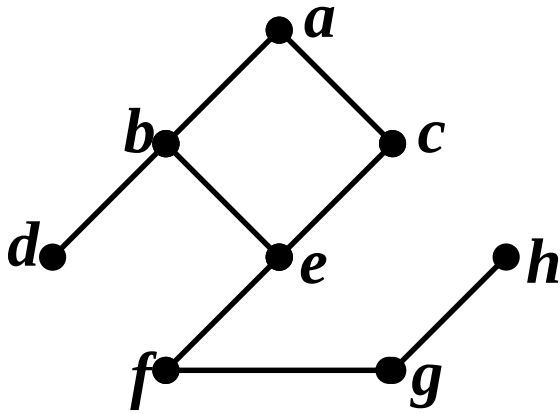
• 너비 우선 탐색(BFS: Breath First Search)

- ① 시작점 v 를 탐색한다.
- ② 꼭짓점 v 에 인접한 꼭짓점들 $\{v_{s_1}, v_{s_2}, \dots, v_{s_m}\}$ 을 모두 탐색한다.
- ③ 꼭짓점 v_{s_1} 을 v 로 하여 ②를 반복한다. 마찬가지로 나머지 $v_{s_2}, \dots, v_{s_m}$ 에 대해서도 차례대로 적용한다.
- ④ 그래프의 모든 꼭짓점을 탐색할 때까지 ③을 반복한다.

5-3) 너비 우선 탐색-2

- 너비 우선 탐색(BFS) $|V|=8$

- $a > b - c; (b) > d - e, (c); (d), (e) > f; (f) > g; (g) > h$



- $a-b-c-d-e-f-g-h$